Leitura guiada do `app.py`

Interface Streamlit — lugar das raízes e discretização

PSC — Controle Linear

14 de outubro de 2026

Resumo

Este documento percorre, *trecho por trecho*, o arquivo `app.py` (631 linhas). Para cada bloco são ditos três coisas: **o que faz**, **qual API do Streamlit usa** e **qual é a lógica**. As listagens são inseridas direto do arquivo-fonte com `\lstinputlisting`, então o relatório nunca diverge do código real.

Sumário

1 Visão geral 2

2 Imports, configuração e constantes 2

3 Helpers de formatação 4

4 Cabeçalho e escolha do controlador 6

5 Seção 1 — Parâmetros de desempenho 7

6 Seção 2 — Planta 8

7 Seção 3 — Propriedades de $G(s)H(s)$ 9

8 Seção 4 — Passos 11 e 12 do LGR 10

9 Seção 5 — Síntese do controlador 11

10 Seção 6 — Malha fechada 14

11 Seção 7 — Discretização 14

12 Rodapé 15

13 Padrões de sintaxe usados no arquivo 16

14 Onde a interface meets a matemática 16

1 Visão geral

O `app.py` é *só interface*. Nenhuma matemática é feita aqui: cada cálculo é delegado a uma função de `lib.py`, e o que volta é exibido. Essa separação é o que permite rodar a lógica no `AppTest` sem navegador.

Modelo mental do Streamlit. O script é executado *de cima para baixo a cada interação*. Não existe laço nem estado implícito: quando o usuário mexe num campo, o arquivo inteiro roda de novo. Por isso o app usa dois mecanismos de estado:

- `st.form` — agrupa campos e só dispara o *rerun* quando o usuário clica em *Confirmar* (evita recálculo a cada tecla).
- `st.session_state` — dicionário que persiste entre reruns, usado para transportar o polo desejado e os coeficientes da planta entre seções.

Fluxo das seções.

Linha	Bloco	Função
205	1. Parâmetros de desempenho	escolhe o polo dominante s_i por três caminhos
281	2. Planta	monta $G(s)$ e $H(s)$ por coeficientes
325	3. Propriedades de $G(s)H(s)$	FT, zeros, polos, ganho e plano-s
353	4. Passos 11 e 12	tabela de módulo e ângulo em s_i
391	5. Síntese do controlador	calcula $C(s)$, mostra K_c, φ, K_p, K_i, K_d
547	6. Malha fechada	raízes de $1 + CGH = 0$ e teste de estabilidade
575	7. Discretização	$G_d(z)$, $C_d(z)$, $T_d(z)$

2 Imports, configuração e constantes

2.1 Bloco de imports (linhas 18–41)

```
"""
app.py --- Interface Streamlit do projeto de controladores pelo lugar das raízes.

Fluxo da aplicação
------------------
1. Escolha do tipo de controlador (P, PI, PD, PID ou Genérico).
2. Definição do polo desejado por três caminhos alternativos:
   M_p/tau, w_n/xi ou coordenada complexa.
3. Definição da planta G(s) e H(s) por coeficientes.
4. Propriedades da planta em malha aberta (zeros, polos, ganho) e tabelas do LGR.
5. Síntese do controlador (condições de ângulo e módulo).
6. Análise de malha fechada (polos resultantes).
7. Discretização do sistema (cap. 5 do SisContr).

Toda a matemática vive em 'lib.py'; aqui apenas exibimos os resultados.
"""
```

```

import matplotlib.pyplot as plt
import sympy as sp
import streamlit as st

import lib
from lib import (
    DESCRICAO_METODOS,
    coeficientes_ft,
    calcular_controlador,
    calcular_zc,
    calcular_controlador_gen,
    discretizar_sistema,
    ganho_planta,
    get_ft,
    grandezas_polo,
```

```
    pole_from_complex,
    pole_from_freq,
    pole_from_Mp,
    polos_malha_fechada,
    s,
    tabela_lgr,
    z,
)
from sympy.physics.control import pole_zero_numerical_data
```

Sintaxe.

- `import lib` — importa o módulo inteiro, para chamar `lib.parametros_padrao` e `lib._expressao_s` com o prefixo.
- `from lib import (...)` — *selective import*: só os nomes usados com frequência, sem prefixo.
- Os dois estilos convivem de propósito: o primeiro deixa explícito "isto vem do módulo de cálculo", o segundo deixa o corpo do script mais limpo.

Lógica. `pole_zero_numerical_data` do SymPy é a função que extrai os polos e zeros numéricos de uma `TransferFunction` pronta para exibir.

2.2 Configuração e constantes (linhas 43–77)

```
st.set_page_config(page_title="Sintonização de controladores", layout="wide")

TIPOS = ["P", "PI", "PD", "PID", "Genérico"]


# --- helpers ---

st.html(
    """
<style>
/* O Streamlit forca .katex-display{display:inline;margin:0} e isso impede
   qualquer centralizacao, entao a regra precisa de !important. Vale so para a
   FT (container com key="ft"), para nao mexer nas outras equacoes. */
.st-key-ft .katex-display {
    display: block !important;
    text-align: center !important;
    margin: .4rem 0 !important;
    white-space: normal !important;
}
.st-key-ft .katex { font-size: 1.45em !important; }

[data-testid="stMetricValue"] { font-size: 1.15rem !important; }
[data-testid="stMetricLabel"] { font-size: .78rem !important; }
</style>
"""
)

CASAS = 4   # casas decimais exibidas

# substituição s = ... de cada método (SisContr.pdf, cap. 5, sec. 5.2)
FORMULA_DISCRETIZACAO = {
    "euler": r"s \;\approx\; \frac{z - 1}{T_s}",
    "backward": r"s \;\approx\; \frac{z - 1}{z\,T_s}",
    "trapezoidal": r"s \;\approx\; \frac{2}{T_s}\,\frac{z - 1}{z + 1}",
}
```

Sintaxe. `st.set_page_config(...)` *precisa ser a primeira linha executada* do script; `layout="wide"` ocupa a largura toda; **string bruta** `r"..."` para as f-strings do LaTeX (sem `r`, o `\a` do `\approx` viraria um *bell*);

Lógica.

- `TIPOS` alimenta o `selectbox` do tipo de controlador.
- O bloco `st.html(...)` injeta um `<style>`. O comentário no código registra *por que* o `!important` é obrigatório: o Streamlit sobrescreve `.katex-display` com `display:inline;margin:0`, e

`display:inline` anula tanto a centralização quanto o tamanho do KaTeX. O seletor `.st-key-ft` limita o efeito ao container que tem `key="ft"` (Streamlit gera a classe `st-key-` + a chave), de modo que as outras equações do app não mudam.

- `CASAS` fixa o arredondamento em 4 casas para toda a exibição.
- `FORMULA_DISCRETIZACAO` mostra a substituição de cada método (SisContr.pdf, cap. 5, sec. 5.2) acima do resultado.

3 Helpers de formatação

3.1 `num` (linhas 80–87)

```
1
2  def num(v, casas=CASAS):
3      """Número sem zeros à direita: 0.50000 -> 0.5, 3.0000 -> 3."""
4      q = round(float(v), casas)
5      if q == 0:
6          return "0"
7      if q == int(q) and abs(q) < 1e15:
8          return str(int(q))
9      return f"{q:.{casas}f}".rstrip("0").rstrip(".")
```

Sintaxe. `round()`; `f"q:.4f"` (specification de formatação); `.rstrip("0").rstrip(".")` remove zeros e ponto finais.

Lógica. Ponto único de arredondamento do app. `3.0000` vira `3` e `0.50000` vira `0.5` — nada de zeros à direita na tela. O teste `abs(q) < 1e15` evita que um inteiro gigante saia com notação científica.

3.2 `latex_ft` (linhas 90–121)

```
1
2  def latex_ft(expr, var=s):
3      """LaTeX de N(s)/D(s) com o líder do denominador puxado para fora.
4
5      1/(10000*s**2 - 11772) -> 1 / {10000 (s^2 - 1.1772)}
6      """
7      def _arredonda(e):
8          subs = {}
9          for a in e.atoms(sp.Float, sp.Rational):
10             v = round(float(a), CASAS)
11             if v == 0 and float(a) != 0:        # 1e-5 nao pode virar 0
12                 subs[a] = sp.Float(float(f"{float(a):.3g}"))
13             else:
14                 subs[a] = sp.Integer(v) if v == int(v) else sp.Float(v)
15         return e.xreplace(subs)
16
17     n, d = sp.fraction(sp.cancel(sp.simplify(expr)))
18     if not n.free_symbols and not d.free_symbols:
19         return sp.latex(_arredonda(n / d))
20     if sp.simplify(d) == 1:                      # polinômio puro, sem "/1"
21         return sp.latex(_arredonda(sp.expand(n)))
22     lc = sp.LC(sp.Poly(sp.expand(d), var))
23     interno = _arredonda(sp.expand(sp.expand(d) / lc))
24     # em SymPy Float(1.0) == 1 e falso, entao compara por valor
25     lc1 = not lc.free_symbols and float(lc) == 1
26     int1 = not interno.free_symbols and float(interno) == 1
27     if int1:
28         den_latex = sp.latex(_arredonda(lc))
29     elif lc1:
30         den_latex = sp.latex(interno)
31     else:
32         den_latex = sp.latex(_arredonda(lc)) + r"\left(" + sp.latex(interno) + r"\right)"
33     return r"\frac{" + sp.latex(_arredonda(n)) + "}{" + den_latex + "}"
```

Sintaxe. *Função aninhada* `_arredonda`; `e.atoms(sp.Float, sp.Rational)` localiza todos os números da expressão; `xreplace(substitutions)` troca todos de uma vez (substituição por expressão

arbitrária exige `xreplace`, não `subs`); `not x.free_symbols and float(x) == 1` testa se um coeficiente é a constante 1.

Lógica. Esta função existe por um problema de *forma*, não de matemática. Sem ela, $\frac{1}{10000s^2-11772}$ sai com números crus (11772). Ela arredonda todos os coeficientes e puxa o líder do denominador para fora:

$$\frac{1}{10000s^2 - 11772} \longrightarrow \frac{1}{10000\,(s^2 - 1.1772)}$$

O comentário da linha 112 registra uma armadilha do SymPy: `sp.Float(1.0) == 1` é *Falso*, então a comparação tem de ser por valor via `float()`.

3.3 `ft_para_latex` (linhas 124–129)

```
def ft_para_latex(expr, var=s):
    """ft_para_latex com retorno seguro se o SymPy recusar a expressão."""
    try:
        return latex_ft(expr, var)
    except (sp.PolynomialError, TypeError, ValueError, ZeroDivisionError):
        return sp.latex(sp.simplify(expr))
```

Lógica. Casca de segurança: se o SymPy recusar a expressão no `Poly`, cai para `sp.latex(sp.simplify(...))`. Garante que a tela *nunca* fica sem fórmula.

3.4 `coeficientes_texto` (linhas 135–138)

```

def coeficientes_texto(coef, casas=CASAS):
    """Coeficientes (numerador, denominador) como texto."""
    return tuple(" ".join(num(v, casas) for v in coef[k])
                 for k in ("numerador", "denominador"))
```

Sintaxe. *Generator expression* aninhado em `join`; *destructuring* de tupla `a, b = f(x)`; `tuple(... for k in (...))`.

Lógica. Achata o dicionário de `lib.coeficientes_ft` em dois textos: `"1 84.2791 376.1164"`.

3.5 `complexo_para_latex` (linhas 141–146)

```
def complexo_para_latex(v, casas=CASAS):
    """Complexo como a ± jb; raiz real sai sem parte imaginária."""
    v = complex(v)
    if abs(v.imag) < 10 ** (-casas):
        return num(v.real, casas)
    return f"{num(v.real, casas)} \\pm j\\,{num(abs(v.imag), casas)}"
```

Lógica. Raiz real sai como `-4.75`; raiz complexa sai como `-2 ± j3`. O limiar $10^{-\text{casas}}$ decide o que é "real na precisão de exibição".

3.6 `zero_txt` (linhas 149–162)

```
def zero_txt(posicoes, casas=CASAS):
    """Zeros como o PDF escreve: 'z = 8.66' (o parametro de 'Gc = Kc(s+z)').

    O zero fica em 's = -z'; mostramos os dois para nao confundir.
    Zeros iguais aparecem uma vez, com a multiplicidade.
    """
    vistos = {}
    for p in posicoes:
        vistos[round(float(p), casas)] = vistos.get(round(float(p), casas), 0) + 1
    partes = []
    for p, mult in vistos.items():
        txt = f"z = {num(p, casas)} \\ (s = {num(-p, casas)})"
        partes.append(f"{txt} \\times {mult}" if mult > 1 else txt)
    return ", ".join(partes)
```

Sintaxe. `dict.get(chave, 0) + 1` conta ocorrências; *conditional expression* dentro de f-string.

Lógica. O zero do controlador vive em $s = -z$, e o PDF da disciplina escreve o parâmetro z. Por isso a função mostra *os dois*: `z = 8.6613 (s = -8.6613)`. Zeros coincidentes são impressos uma vez com a multiplicidade ($\times 2$).

3.7 `lista_para_latex` (linhas 165–177)

```python
def lista_para_latex(valores, casas=CASAS):
    """Lista de zeros/polos; par conjugado aparece uma vez só."""
    vistos = set()
    partes = []
    for v in valores:
        v = complex(v)
        chave = (round(v.real, casas), round(abs(v.imag), casas))
        if abs(v.imag) > 10 ** (-casas):
            if chave in vistos:
                continue
            vistos.add(chave)
        partes.append(complexo_para_latex(v, casas))
    return ", ".join(partes)
```

Sintaxe. `set` como estrutura de comparação; tupla como chave de dicionário `(round(real), round(abs(imag)))`.

Lógica. Mesma ideia do helper anterior, mas para a lista completa de zeros e polos: o par conjugado $a \pm jb$ aparece uma única vez.

3.8 `tentar` (linhas 180–185)

```python
def tentar(acao, contexto=""):
    """(True, resultado) ou (False, mensagem de erro)."""
    try:
        return True, acao()
    except (ValueError, TypeError, ZeroDivisionError) as exc:
        return False, f"{contexto}{exc}" if contexto else str(exc)


# --- 1. Controlador ---
```

Sintaxe. Função que *recebe outra função* (*higher-order*); tupla de retorno `(True, valor)` ou `(False, mensagem)`.

Lógica. Traduz exceções em mensagens de tela. É o que permite escrever

```
ok, ft = tentar(lambda: get_ft(...), "Planta inválida: ")
```

com uma linha, em vez de um `try/except` de seis linhas em cada seção. O prefixo `contexto` diz *qual* etapa falhou.

4 Cabeçalho e escolha do controlador

4.1 (linhas 190–200)

```python
st.title("Sintonia de controladores pelo método do lugar das raízes")

st.caption(
    "Método do LGR (SisContr.pdf, cap. 2 e 4) com discretização opcional "
    "(SisContr.pdf, cap. 5)."
)

control_type = st.selectbox("Tipo de controlador", TIPOS, key="control_type")

# nomes curto para as fórmulas padrão
MAPA_TIPO = {"P": "P", "PI": "PI", "PD": "PD", "PID": "PID", "Genérico": "GEN"}


# --- 2. Polo desejado ---
```

Sintaxe. `st.title`/`st.caption`; `st.selectbox(...)` devolve o item selecionado e aceita `key=` (identificador estável do widget).

Lógica. `MAPA_TIPO` traduz o rótulo da interface (“Genérico”) para o nome interno usado em `lib` (“GEN”), que por sua vez é o que `parametros_padrao` entende.

5 Seção 1 — Parâmetros de desempenho

5.1 (linhas 205–276)

```
with st.expander("1. Definir parâmetros de desempenho", expanded=True):
    tab_mp, tab_wn, tab_cx = st.tabs(
        [r"$M_P$ [%] e $\tau$ [s]", r"$\omega_n$ e $\xi$", r"$s = \sigma \pm j\omega_d$"]
    )

    pole = st.session_state.get("pole")

    with tab_mp:
        with st.form("form_mp"):
            c1, c2 = st.columns(2)
            M_p = c1.number_input("Sobressinal máximo [%]:", min_value=0.0, max_value=100.0,
                                  value=10.0, step=0.5, format="%.2f")
            tol = c2.radio("Tolerância [%]:", [2, 5], horizontal=True)
            tau = st.number_input("Tempo de acomodação [s]:", min_value=0.0,
                                  value=1.0, step=0.1, format="%.3f")

            st.latex(
                r"\xi=\sqrt{\frac{\ln^2(M_p/100)}{\pi^2+\ln^2(M_p/100)}}"
                r"\qquad\omega_n=\frac{4}{\tau\xi}\ (2\%)\ \text{ou}\ "
                r"\omega_n=\frac{3}{\tau\xi}\ (5\%)"
            )
            if st.form_submit_button("Confirmar"):
                ok, res = tentar(lambda: pole_from_Mp(M_p, tol, tau), "Polo inválido: ")
                if ok:
                    st.session_state["pole"] = res
                    st.success(f"Polo desejado: $s_i = {complexo_para_latex(res)}$")
                else:
                    st.error(res)

    with tab_wn:
        with st.form("form_wn"):
            c1, c2 = st.columns(2)
            omega_n = c1.number_input(r"Frequência natural $\omega_n$ [rad/s]:",
                                      min_value=0.0, value=4.0, step=0.1, format="%.3f")
            xi = c2.number_input(r"Amortecimento $\xi$:", min_value=0.0, max_value=0.999,
                                 value=0.5, step=0.01, format="%.3f")
            st.latex(r"\omega_d=\omega_n\sqrt{1-\xi^2},\quad \sigma=\xi\omega_n")
            if st.form_submit_button("Confirmar"):
                ok, res = tentar(lambda: pole_from_freq(omega_n, xi), "Polo inválido: ")
                if ok:
                    st.session_state["pole"] = res
                    st.success(f"Polo desejado: $s_i = {complexo_para_latex(res)}$")
                else:
                    st.error(res)

    with tab_cx:
        with st.form("form_cx"):
            c1, c2 = st.columns(2)
            real = c1.number_input("Parte real $\\sigma$ (posicione $\\sigma < 0$):",
                                   value=-2.0, step=0.1, format="%.3f")
            imag = c2.number_input("Parte imaginária $\\omega_d$:",
                                   value=3.0, step=0.1, format="%.3f")
            st.latex(r"s_i=\sigma\pm j\omega_d,\qquad \sigma=-\operatorname{Re}(s_i)")
            if st.form_submit_button("Confirmar"):
                ok, res = tentar(lambda: pole_from_complex(real, imag), "Polo inválido: ")
                if ok:
                    st.session_state["pole"] = res
                    st.success(f"Polo desejado: $s_i = {complexo_para_latex(res)}$")
                else:
                    st.error(res)
```

```
pole = st.session_state.get("pole")
```

```
2
3  if pole is None:
4      st.error("**Defina os parâmetros de desempenho para continuar.**")
5      st.stop()
6
7  sigma, w_d = grandezas_polo(pole)
8  st.markdown(
9      f"**Polo desejado:** $s_i = {complexo_para_latex(pole)}$    "
10     f"$\\sigma = {num(sigma)}$, $\\omega_d = {num(w_d)}$"
11 )
12
13
14 # --- 3. Planta ---
```

Sintaxe.

- `with st.expander(...)` — painel recolhível; `expanded=True` abre por padrão.
- `st.tabs([...])` — abas; cada item é o *rótulo*, que aceita LaTeX inline entre $$.
- `with st.form(...)` + `st.form_submit_button` — *batch*: os widgets só disparam *rerun* no clique.
- `st.columns(2)` retorna uma lista; `c1, c2 = st.columns(2)` *desempacota* — cada elemento é um contêiner independente com seus próprios `st.*`.
- `st.number_input(min_value=, max_value=, value=, step=, format=)`: `format="%.2f"` fixa duas casas na edição.
- `st.radio(..., horizontal=True)`.
- `st.latex(...)` renderiza KaTeX em modo *display*.
- `st.session_state["pole"] = res` grava; `st.session_state.get("pole")` lê sem KeyError; `.get(k)` devolve `None` se ainda não existe.
- `st.stop()` **interrompe a execução do script**.

Lógica. Três caminhos mutuamente exclusivos para obter o polo dominante, todos Reductiondo ao mesmo formato:

1. $(M_P,\ \tau) \longrightarrow$ `pole_from_Mp`
2. $(\omega_n,\ \xi) \longrightarrow$ `pole_from_freq`
3. $(\sigma,\ \omega_d) \longrightarrow$ `pole_from_complex`

O ponto de controle do script é a linha 268:

```
pole = st.session_state.get("pole")    if pole is None: st.error(...); st.stop()
```

Sem `st.stop()`, o script continuaria e quebraria na seção 2 com um `TypeError` em `None`. É a *guarda* do programa.

6 Seção 2 — Planta

6.1 (linhas 281–313)

```
1  with st.expander("2. Definir a planta", expanded=True):
2      if "TF" not in st.session_state:
3          st.session_state["TF"] = {"g_num": "1", "g_den": "1, 8, 32, 0",
4                                    "h_num": "1", "h_den": "1, 4"}
5
6      with st.form("form_planta"):
7          col_g, col_h = st.columns(2)
8          with col_g:
```

```
            st.subheader("$G(s)$")
            g_num = st.text_input("Coeficientes do numerador de $G(s)$ (maior → menor grau):",
                                  key="tf_g_num", value=st.session_state["TF"]["g_num"])
            g_den = st.text_input("Coeficientes do denominador de $G(s)$ (maior → menor grau):",
                                  key="tf_g_den", value=st.session_state["TF"]["g_den"])
        with col_h:
            st.subheader("$H(s)$")
            h_num = st.text_input("Coeficientes do numerador de $H(s)$ (maior → menor grau):",
                                  key="tf_h_num", value=st.session_state["TF"]["h_num"])
            h_den = st.text_input("Coeficientes do denominador de $H(s)$ (maior → menor grau):",
                                  key="tf_h_den", value=st.session_state["TF"]["h_den"])

        st.form_submit_button("Confirmar planta")

    st.session_state["TF"] = {"g_num": g_num, "g_den": g_den,
                              "h_num": h_num, "h_den": h_den}

    if not all(str(v).strip() for v in (g_num, g_den, h_num, h_den)):
        st.error("Preencha todos os coeficientes da planta.")
        st.stop()

    ok, ft = tentar(lambda: get_ft(g_num, g_den, h_num, h_den), "Planta inválida: ")
    if not ok:
        st.error(ft)
        st.stop()


# --- 4. Propriedades de G(s)H(s) ---
```

Sintaxe. `if "TF"not in st.session_state: ...` (inicialização preguiçosa); `st.subheader`; `all(...)` com gerador para validar vários campos de uma vez; `str(v).strip()` normaliza.

Lógica. Os quatro campos de texto recebem coeficientes do maior para o menor grau (`"1, 8, 32, 0"`). Três guardas, na ordem:

1. algum campo vazio → erro;
2. `get_ft` levanta exceção → erro e `stop`;
3. caso contrário, `ft` segue para as seções seguintes.

7 Seção 3 — Propriedades de $G(s)H(s)$

7.1 (linhas 318–350)

```
→→→→
zeros, polos = pole_zero_numerical_data(ft)
ganho = ganho_planta(ft)
tbl = tabela_lgr(pole, polos, zeros)

col1, col2 = st.columns([1, 1])

with col1:
    with st.expander("3. Propriedades de $G(s)H(s)$", expanded=True):
        st.latex(f"G(s)H(s)={ft_para_latex(ft.doit().to_expr())}")
        st.markdown("**Zeros**")
        st.markdown(f"${lista_para_latex(zeros)}$" if zeros else "Não há zeros.")
        st.markdown("**Polos**")
        st.markdown(f"${lista_para_latex(polos)}$" if polos else "Não há polos.")
        st.markdown(f"**Ganho** (razão dos coeficientes líderes): $K = {num(ganho)}$")

        # plano-s
        fig, ax = plt.subplots(figsize=(4.2, 3.2))
        ax.axhline(0, color="grey", lw=0.7)
        ax.axvline(0, color="grey", lw=0.7)
        if zeros:
            ax.plot([complex(z).real for z in zeros], [complex(z).imag for z in zeros],
                    "o", color="tab:green", label="zeros")
        if polos:
            ax.plot([complex(p).real for p in polos], [complex(p).imag for p in polos],
                    "x", color="tab:red", ms=9, mew=2, label="polos")
        ax.plot(pole.real, pole.imag, "*", color="black", ms=16, label=r"$s_i$")
```

```
        ax.set_xlabel("Re(s)")
        ax.set_ylabel("Im(s)")
        ax.legend(fontsize=7)
        ax.grid(alpha=0.3)
        fig.tight_layout()
        st.pyplot(fig)
        plt.close(fig)
```

Sintaxe. *Desempacotamento múltiplo*: `zeros, polos = pole_zero_numerical_data(ft)`; `st.columns([1,1])` com *ratios* de largura; `with col1: ...` entra no contêiner da coluna; `ax.axhline/axvline` desenha os eixos; *conditional expression* montando as listas de coordenadas; `st.pyplot(fig)` e `plt.close(fig)`.

Lógica. Três chamadas devolvem toda a informação da planta: zeros, polos e ganho. O plano-s é um `matplotlib` puro: círculo verde para zeros, `x` vermelho para polos, estrela preta em s_i, eixos em cinza. `fig.tight_layout()` evita que os rótulos dos eixos sejam cortados.

8 Seção 4 — Passos 11 e 12 do LGR

8.1 (linhas 352–386)

```
→→→→
with col2:
    with st.expander("4. Passos 11 e 12 do LGR", expanded=True):
        st.markdown(
            "**Passo 11 — distâncias e produtos** "
            r"(condição de módulo: $1+K\prod_j(s_i-z_j)/\prod_k(s_i-p_k)=0$)."
        )
        # zeros: índice j, polos: índice k
        for rotulo, simbolo, indice, linhas in (("zeros", "z", "j", tbl["zeros"]),
                                                ("polos", "p", "k", tbl["polos"])):
            if not linhas:
                st.markdown(f"Não há {rotulo}.")
                continue
            st.markdown(f"*{rotulo.capitalize()}*")
            for i, linha in enumerate(linhas):
                pos = complex(linha["posicao"])
                sinal = "-" if pos.imag < 0 else "+"
                local = (f"{num(pos.real)}" if abs(pos.imag) < 1e-12
                         else f"{num(pos.real)} {sinal} j\\,{num(abs(pos.imag))}")
                st.markdown(
                    f"- ${simbolo}_{{{i}}}$ = ${local}$: "
                    f"$|s_i - {simbolo}_{indice}| = {num(linha['modulo'])}$, "
                    f"$\\angle(s_i - {simbolo}_{indice}) = {num(linha['fase'])}^\\circ$"
                )

        st.latex(rf"\prod_j |s_i - z_j| = {num(tbl['produto_zeros'])}")
        st.latex(rf"\prod_k |s_i - p_k| = {num(tbl['produto_polos'])}")

        st.markdown("**Passo 12 — soma das fases**")
        st.latex(r"\sum_j \angle(s_i - z_j) - \sum_k \angle(s_i - p_k)")
        st.latex(f"= {num(tbl['angulo_ponto'])}^\\circ")
        st.markdown("**Déficit de ângulo do controlador**")
        st.latex(
            rf"\varphi = 180^\circ - \left({num(tbl['angulo_ponto'])}^\circ\right)"
            rf" = {num(tbl['phi'])}^\circ"
        )


# --- 5. Síntese do controlador ---
```

Sintaxe. *Tupla de agrupamento* `for rotulo, simbolo, indice, linhas in ((...),(...))` gera um laço duplo sem repetir código; `f-string` r (crua) para o LaTeX que contém `\`; `st.latex(rf"...")` concatena o valor calculado ao LaTeX fixo; `list comprehension` com `enumerate`.

Lógica. Percorre a tabela devolvida por `tabela_lgr` e imprime, para cada z_j e p_k:

$$\|s_i - z_j\| \quad \text{e} \quad \angle(s_i - z_j)$$

Depois fecha com os produtos (passo 11, condição de módulo), a soma das fases e o déficit φ (passo 12, condição de ângulo). É exatamente a sequência do PDF da disciplina.

9 Seção 5 — Síntese do controlador

9.1 Opções específicas (linhas 391–421)

```
→→→→──
with st.expander("5. Síntese do controlador", expanded=True):
    tipo = MAPA_TIPO[control_type]

    opcoes_generico = {}
    if control_type == "PID":
        opcoes_generico["relacao_zeros"] = st.number_input(
            "Razão entre os dois zeros ($z_2 = r\\,z_1$); $r=1$ mantém os zeros coincidentes:",
            min_value=0.0, value=1.0, step=0.1, format="%.3f",
        )
    if control_type == "Genérico":
        gz = st.columns(3)
        n_zeros = gz[0].number_input("Zeros informados (z > 0):", min_value=0, value=1, step=1)
        n_polos = gz[1].number_input("Polos a calcular:", min_value=1, value=1, step=1)
        n_fixos = gz[2].number_input("Polos fixos:", min_value=0, value=0, step=1)

        c1, c2, c3 = st.columns(3)
        zeros_ctrl = [
            c1.number_input(f"Zero {i + 1} (em $s=-z$):", min_value=0.0,
                            value=1.0 + i, step=0.5, format="%.3f", key=f"gz{i}")
            for i in range(int(n_zeros))
        ]
        relacoes_polos = [
            c2.number_input(f"Razão polo {i + 1}/polo 1:", min_value=0.0,
                            value=1.0, step=0.1, format="%.3f", key=f"gr{i}")
            for i in range(int(n_polos))
        ]
        polos_fixos = [
            c3.number_input(f"Polo fixo {i + 1} (0 = integrador):", value=0.0,
                            step=0.5, format="%.3f", key=f"gp{i}")
            for i in range(int(n_fixos))
        ]
```

Sintaxe. `st.__dict__` é um dicionário comum; *compreensões de lista* com `range()` e *f-string* no `key=` (`key=f"gz{i}"`) — o `key` precisa ser único, senão o Streamlit levanta `StreamlitDuplicateElementId`.

Lógica. Só o PID e o Genérico precisam de entrada extra:

- **PID**: razão r entre os dois zeros ($z_2 = r z_1$); $r = 1$ mantém os zeros coincidentes, que é o PID clássico.
- **Genérico**: quantos zeros informar, quantos polos calcular e quantos polos fixos (sendo 0 o integrador).

9.2 A função `_calcular` (linhas 423–478)

```
→→→→──
    st.markdown(f"**Controlador: {control_type}**")

    # --- cálculo ---
    def _calcular():
        """Reproduz as células PI/PD/PID/GEN do notebook 'pid.ipynb'."""
        avisos = []

        # integrador do PI/PID
        polos_int = list(polos)
        if tipo in ("PI", "PID"):
            polos_int = polos_int + [0j]

        if tipo == "P":
            # P: sem zero, só a condição de módulo
            return calcular_controlador(
                ft, pole, polos, zeros, sigma, w_d,
                n_zeros=0, relacoes=[], polos_denominador=(),
                avisos=avisos,
            ), avisos
```

```
21
22          if tipo == "GEN":
23              return calcular_controlador_gen(
24                  ft, pole, polos, zeros, sigma, w_d,
25                  zeros_controlador=zeros_ctrl,
26                  n_polos=int(n_polos),
27                  relacoes_polos=relacoes_polos,
28                  polos_fixos=polos_fixos,
29                  avisos=avisos,
30              ), avisos
31
32          if tipo == "PID":
33              # PID: dois zeros com razão configurável
34              r = float(opcoes_generico["relacao_zeros"])
35              return calcular_controlador(
36                  ft, pole, polos_int, zeros, sigma, w_d,
37                  n_zeros=2, relacoes=[1.0, r],
38                  polos_denominador=[0.0],
39                  avisos=avisos,
40              ), avisos
41
42          # PI e PD via calcular_zc
43          pid, pd_, pi_ = tipo == "PID", tipo == "PD", tipo == "PI"
44          zc, Kc = calcular_zc(ft, pole, polos_int, zeros, ganho, w_d, sigma,
45                               pid, pd_, pi_)
46          return calcular_controlador(
47              ft, pole, polos_int, zeros, sigma, w_d,
48              n_zeros=1, relacoes=[1.0],
49              polos_denominador=[0.0] if (pid or pi_) else [],
50              avisos=avisos,
51          ), avisos
52
53      ok, saida = tentar(_calcular, "Falha no projeto: ")
54      if not ok:
55          st.error(saida)
56          st.stop()
57      resultado, avisos = saida
58
59      Gc = resultado["Gc"]
60      Kc = resultado["ganho"]
61      phi_ctrl = resultado["phi"]
```

Sintaxe. *Função aninhada* que fecha as variáveis do script (*closure*); `return x, avisos` devolvendo *tupla*; lista `polos_int = list(polos)` copiada antes de `.append`.

Lógica. Cada ramo monta os argumentos do `lib` e devolve o mesmo formato `(resultado, avisos)`, de modo que a interface não precisa saber qual controlador foi escolhido:

Tipo	Zeros	Polos
P	nenhum ($n = 0$)	nenhum
PI	1	0 (integrador)
PD	1	nenhum
PID	2, com razão r	0 (integrador)
Genérico	informados	calculados + fixos

O detalhe do integrador está nas linhas 431–433: para PI e PID o polo 0 é *acrescentado* a `polos_int` antes da chamada, porque `calcular_zc` do notebook já espera a lista com o integrador incluído. PI e PD passam por `calcular_zc` (assinatura do `pid.ipynb`) e depois por `calcular_controlador`, que é quem produz o dicionário completo.

9.3 Avisos e cartões de resultado (linhas 484–507)

```
1   →→→→——
2   for aviso in avisos:
3       if aviso.startswith("Aviso:"):
4           st.warning(aviso)
5
6   # --- linha de resultados ---
7   Kp = Ki = Kd = None
```

```
if tipo != "GEN":
    Kp, Ki, Kd = lib.parametros_padrao(Gc, tipo)
elif sp.simplify(sp.fraction(sp.together(lib._expressao_s(Gc)))[1]) in (1, s):
    num_gc, den_gc = sp.fraction(sp.together(lib._expressao_s(Gc)))
    pol = sp.Poly(sp.expand(num_gc), s)
    c0 = float(pol.coeff_monomial(1))
    c1 = float(pol.coeff_monomial(s))
    c2 = float(pol.coeff_monomial(s ** 2))
    if sp.simplify(den_gc) == s:
        Kp, Ki, Kd = c1, c0, c2
    else:
        Kp, Ki, Kd = c0, 0.0, c1

itens = [("Ganho $K_c$", num(Kc)), (r"Ângulo $\varphi$", f"{num(phi_ctrl)}°")]
itens += [(n, num(v)) for n, v in (("$K_p$", Kp), ("$K_i$", Ki), ("$K_d$", Kd))
          if v is not None]
for coluna, (rotulo, valor) in zip(st.columns(len(itens)), itens):
    coluna.metric(rotulo, valor)
```

Sintaxe. `if aviso.startswith("Aviso:")` separa aviso de resultado (o `_avisar` do `lib` emite mensagens normais e avisos); `zip(st.columns(n), itens)` emparelha contêineres e dados; *list comprehension* com *if de filtro* `if v is not None`.

Lógica. Cada mensagem que começa com `"Aviso:"` vira `st.warning` (caixa amarela); o resto é saída normal de cálculo e não é mostrado. Os cartões K_c, φ, K_p, K_i, K_d só aparecem os que fazem sentido para o tipo escolhido.

A linhas 490–501 cobrem o caso do Genérico: quando o denominador é 1 ou s, a expressão ainda é da forma PI/PD/PID e os coeficientes podem ser lidos.

9.4 Singularidades e função de transferência (linhas 509–542)

```
# --- singularidades ---
zeros_ctrl = zero_txt(resultado["zeros"])
if tipo == "GEN":
    calc = ", ".join(f"$s = {num(-p)}$" for p in resultado["polos_calculados"])
    fixos = resultado["polos"][len(resultado["polos_calculados"]):]
    polos_ctrl = f"calculados: {calc}"
    if fixos:
        polos_ctrl += " | fixos: " + ", ".join(f"$s = {num(p)}$" for p in fixos)
elif resultado["polos"]:
    polos_ctrl = ", ".join(f"$s = {num(p)}$" for p in resultado["polos"])
else:
    polos_ctrl = "nenhum"

st.markdown(f"**Zeros:** ${zeros_ctrl or 'nenhum'}$  |  **Polos:** {polos_ctrl}")

if tipo == "P":
    st.warning(
        "O controlador **P** não possui zero nem polo: ele **não altera a fase**. "
        "Apenas a condição de módulo é satisfeita — confirme nos passos 11/12 se "
        "$s_i$ já pertence ao LGR da planta pura."
    )

# --- função de transferência ---
cabecalho = st.columns([6, 1])
cabecalho[0].markdown("**Função de transferência**")
with cabecalho[1]:
    with st.popover("Coeficientes"):
        coef = resultado["coeficientes"]
        cn, cd = coeficientes_texto(coef)
        st.markdown(f"Numerador: ${cn}$")
        st.markdown(f"Denominador: ${cd}$")

with st.container(key="ft"):
    st.markdown(f"$$G_c(s) = {ft_para_latex(Gc)}$$")
```

Sintaxe. *Slicing* `resultado["polos"][n:]` para separar calculados de fixos; `with cabecalho[1]:` entra na segunda coluna; `st.popover("...")` cria o botão que abre os coeficientes; `with st.container(key="ft"):` gera a classe CSS `st-key-ft`.

Lógica. Os zeros aparecem como $z = \ldots$ ($s = -z$), no mesmo formato do PDF. O Genérico

separa visualmente polos *calculados* de *fixos*. A função de transferência é o único bloco com container nomeado, porque é o único que recebe o tratamento especial de centralização e tamanho do LaTeX.

10 Seção 6 — Malha fechada

10.1 (linhas 545–570)

```
→→→→——
# --- 6. Malha fechada ---

with st.expander("6. Polos de malha fechada ($1 + C(s)G(s)H(s) = 0$)"):
    try:
        fechados = polos_malha_fechada(ft, Gc)
        c1, c2 = st.columns(2)
        with c1:
            st.markdown("**Polos de malha fechada**")
            for i, r in enumerate(fechados):
                v = r["valor"]
                sinal = "-" if v.imag < 0 else "+"
                local = (f"{num(v.real)}" if abs(v.imag) < 1e-12
                         else f"{num(v.real)} {sinal} j\\,{num(abs(v.imag))}")
                st.markdown(f"- $p_{{{i}}} = {local}$")
        with c2:
            estavel = all(r["valor"].real < 0 for r in fechados)
            if estavel:
                st.success("Sistema **estável** (todos os polos no semiplano esquerdo).")
            else:
                st.error("Sistema **instável**: há polo no semiplano direito.")
            st.markdown(
                f"Polo desejado presente: "
                f"{'sim' if any(abs(r['valor'] - pole) < 1e-6 for r in fechados) else 'não'}"
            )
    except Exception as exc:  # noqa: BLE001
        st.warning(f"Não foi possível calcular os polos de malha fechada: {exc}")


# --- 7. Discretização ---
```

Sintaxe. `try/except Exception` com `# noqa: BLE001` para silenciar o linter; *generator* com `all(... for r in fechados)`; `any(abs(r["valor"] - pole) < 1e-6 ...)` para checar se o polo desejado está na lista.

Lógica. Calcula as raízes de $1 + C(s)G(s)H(s) = 0$ e julga a estabilidade pelo critério de Routh: sistema estável se *todos* os polos tiverem $\mathrm{Re}(p) < 0$. O `except` largo é deliberado — falha de raiz quadrada emDegree alto não pode derrubar a página.

11 Seção 7 — Discretização

11.1 (linhas 575–624)

```
→→→→——
with st.expander("7. Discretização (cap. 5 – SisContr.pdf)"):
    st.caption(
        "SisContr.pdf, cap. 5, sec. 5.2 (PDF p. 55-57): Euler progressivo, "
        "Euler regressivo e trapezoidal (Tustin)."
    )

    c1, c2 = st.columns(2)
    metodo = c1.selectbox(
        "Método",
        list(DESCRICAO_METODOS),
        format_func=lambda m: DESCRICAO_METODOS[m],
    )
    Ts = c2.number_input("Período de amostragem $T_s$ [s]:",
                         min_value=0.0, value=0.1, step=0.01, format="%.4f")

    st.markdown("Substituição aplicada:")
    st.latex(FORMULA_DISCRETIZACAO[metodo] + rf"\qquad\text{{com }}\; T_s = {Ts}")
```

```
19
20      if Ts <= 0:
21          st.error("O período de amostragem deve ser positivo.")
22      else:
23          ok, disc = tentar(
24              lambda: discretizar_sistema(ft, Gc, Ts, metodo), "Falha na discretização: "
25          )
26          if not ok:
27              st.error(disc)
28          else:
29              cols = st.columns(3)
30              for coluna, titulo, expr in zip(
31                  cols,
32                  (r"Planta $G_d(z)$", r"Controlador $C_d(z)$",
33                   r"Malha aberta $T_d(z)$"),
34                  (disc["planta_discreta"], disc["controlador_discreto"],
35                   disc["sistema_aberto_discreto"]),
36              ):
37                  with coluna:
38                      st.markdown(f"**{titulo}**")
39                      st.latex(ft_para_latex(expr, var=z))
40
41              with st.expander("Coeficientes em $z$"):
42                  for nome, expr in (
43                      ("G_d(z)", disc["planta_discreta"]),
44                      ("C_d(z)", disc["controlador_discreto"]),
45                      ("T_d(z)", disc["sistema_aberto_discreto"]),
46                  ):
47                      c = coeficientes_ft(expr, var=z)
48                      cn, cd = coeficientes_texto(c)
49                      st.markdown(f"**{nome}**")
50                      st.markdown(f"- Numerador: ${cn}$")
51                      st.markdown(f"- Denominador: ${cd}$")
```

Sintaxe. `st.selectbox(..., format_func=lambda ...)` mostra a descrição longa mas grava a chave curta; `zip(cols, titles, exprs)` distribui três resultados em três colunas; *aninhamento* de `st.expander` dentro do expander principal.

Lógica. O usuário escolhe o método e T_s; o app mostra a substituição que será aplicada e devolve as três FTs:

$$G_d(z) = \mathcal{D}[G(s)], \qquad C_d(z) = \mathcal{D}[C(s)], \qquad T_d(z) = C_d(z)\,G_d(z)$$

O detalhe técnico está na chamada `ft_para_latex(expr, var=z)`: o argumento `var` troca o símbolo s por z na hora de montar o polinômio, então o mesmo formatador serve para os dois domínios.

12 Rodapé

12.1 (linhas 627–631)

```
1  →→→→———
2  st.divider()
3  st.caption(
4      "Teoria: *SisContr.pdf* (cap. 2, 4 e 5) e Ogata, *Engenharia de Controle "
5      "Moderno* (cap. 5, 6 e 8)."
6  )
```

Sintaxe. `st.divider()` (linha horizontal) e `st.caption` para a nota de rodapé em texto pequeno.

13 Padrões de sintaxe usados no arquivo

Padrão	Para que serve
`with st.expander`	seção recolhível (a forma de organizar 7 etapas)
`with st.form`	agrupar campos e só recalcular no "Confirmar"
`with st.columns`	layout em colunas; cada elemento é um contêiner
`st.session_state`	levar valores entre seções e entre reruns
`key="..."`	identidade estável do widget *e* do container
`st.stop()`	guardas: abortar antes de um erro em cascata
`tentar(lambda: ...)`	exceção → mensagem na tela
`st.metric`	cartões numéricos de resultado
`st.popover`	detalhe sob demanda (coeficientes)
`st.pyplot`	figuras matplotlib
`st.html`	CSS cru (o Streamlit sanitiza o resto)

14 Onde a interface meets a matemática

Bloco da tela	Função de `lib.py`	Equação
Parâmetros de desempenho	`pole_from_Mp / _freq / _complex`	$s_i = -\sigma \pm j\omega_d$
Planta	`get_ft`, `ganho_planta`	$G(s)H(s)$
Propriedades	`pole_zero_numerical_data`	zeros e polos
Passos 11/12	`tabela_lgr`	$\|s_i - z_j\|$, φ
Síntese	`calcular_controlador`, `_gen`, `calcular_zc`	$C(s)$
Malha fechada	`polos_malha_fechada`	$1 + CGH = 0$
Discretização	`discretizar_sistema`	$s \approx \frac{2}{T_s}\frac{z-1}{z+1}$
Cartões K_p, K_i, K_d	`parametros_padrao`	forma padrão do controlador